Basic question 1: Searching and sorting

Here is a sorted array with 15 integer values:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
10	30	40	50	70	80	90	100	120	140	150	170	180	200	210

a) A binary search is performed, searching for the value 71. Indices low and high are used to specify the search interval. What are the values of low and high after each comparison?

Note: There are more cells than you need here! You should fill in exactly one column per comparison performed by the search. The initial values of high and low are pre-filled.

Number of compared elements:	0	1	2	3	4	5	6	7	8
Low:	0	0	4	4	5				
High:	14	6	6	4	4				

b) Are there any values in the array that can be found using fewer comparisons with a linear search from left to right, than with a binary search? List **all** such values.

10, 30, 40

Basic question 2: Hash tables

The hash table to the right uses linear probing with lazy deletion. The hash table uses modular compression (hash(x) % 10), but the hash function is unknown.

0	
1	P
2	
3	R
4	
5	S
6	T
7	V
8	<deleted>
9	U

a) Which one of these is a possible hash function used to build the table?
Note that this is **not** the order values were added in, just the hash function used.
(Select only one option.)

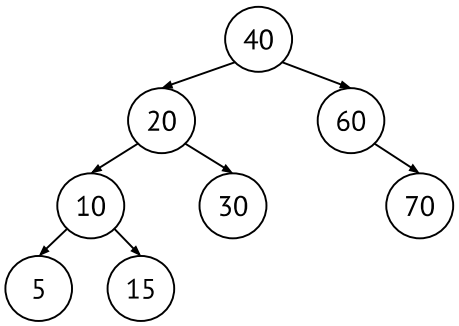
- ☐ hash(P)=31, hash(R)=3, hash(S)=25, hash(T)=17, hash(U)=9
- ☐ hash(P)=42, hash(R)=3, hash(S)=35, hash(T)=56, hash(U)=8
- ☐ hash(P)=11, hash(R)=53, hash(S)=25, hash(T)=36, hash(U)=23
- ☑ hash(P)=21, hash(R)=3, hash(S)=15, hash(T)=6, hash(U)=18
- ☐ hash(P)=11, hash(R)=4, hash(S)=25, hash(T)=66, hash(U)=23
- ☐ hash(P)=21, hash(R)=3, hash(S)=16, hash(T)=7, hash(U)=29

b) Add V with hash(V)=25 to the table (without resizing). You can add the value directly to the table above.

It gets added at index 7 (because both 5 and 6 are occupied), see the table.

Basic question 3: Rebalancing AVL trees

Consider the AVL tree to the right:



a) If we remove the key 70 from the tree (without rebalancing), which node(s) would become unbalanced?
(List all nodes that violate the balance invariant of AVL trees.)

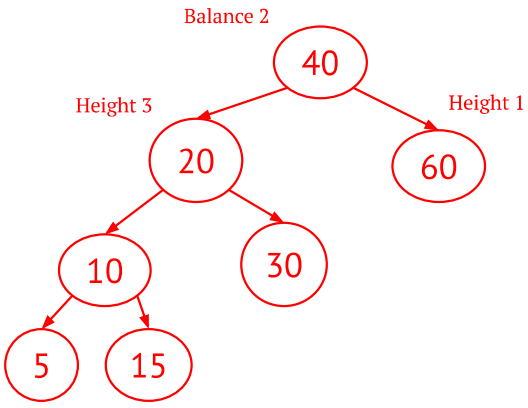
b) Rebalance the tree using tree rotations, restoring the AVL invariant. Use as few rotations as possible. Draw the resulting tree and state which rotations you performed.

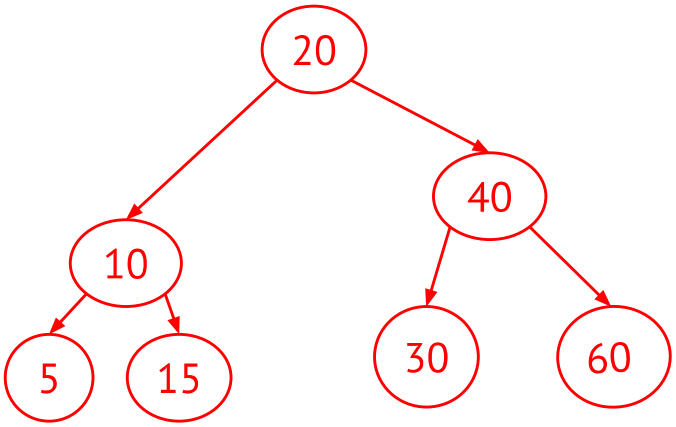
Answer:

Solution:

a) Node 40 is unbalanced.

b) applying a single right-rotation of node 40.





Basic question 4: Heap

Here is a binary minimum-heap with integer values, implemented using a dynamic array:

2	12	5	16

a) The heap needs to be resized to prepare it for inserting a new value. Put the elements in their correct positions in the new, larger array:

2	12	5	16				

b) Show the steps (swaps/array writes) required to insert 1 into the heap (the last array should contain the corrected heap with all five values)

2	12	5	16	1			

2	1	5	16	12			

1	2	5	16	12			

Basic question 5: Complexity

a) Determine the asymptotic complexity of the function f in terms of the input *n* (motivate your answer).

```
f(n):
    k = n
    while k > 1:
        for i from 0 to k · k:
            print(i)        // O(1)
        k = k / 2
```

Hint: The geometric series 1+ ¼ + 1/16 + … adds up to less than 2.

b) Determine the asymptotic complexity of the function g in terms of *n* (motivate your answer):

```
g(n):
    i = 2
    while i · i < n:
        if n mod i = 0:
            print(i)        // O(1)
        i = i + 1
```

Answer (you can also make comments in the code):

Solution:

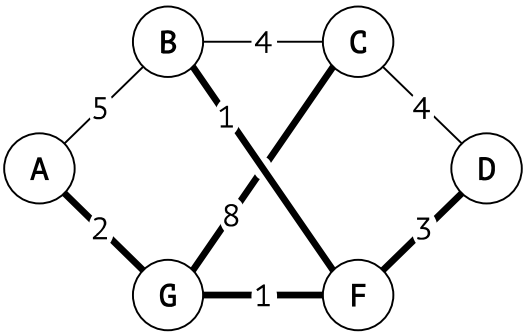
a. The for-loop executes k^2 times, and the while-loop halves k at each iteration. Therefore, we get $n^2 + (n/2)^2 + (n/4)^2 + \ldots$, which is $n^2 + n^2/4 + n^2/16$. Factoring n^2 out, we get $n^2(1 + ¼ + 1/16 + \ldots)$, which is less than $2n^2$. Thus, the time complexity of this function is $O(n^2)$.

   Note: $O(n^2 \log(n))$ is not sharp, but a good enough upper bound to get full points.

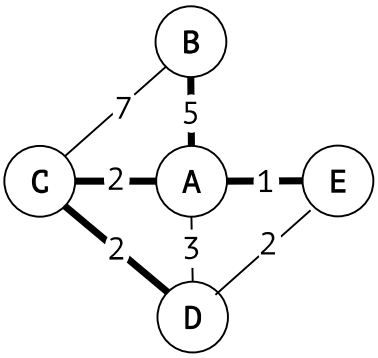
b. The loop condition $i^2 < n$ is equivalent to $i < \sqrt{n}$, so the inner loop has $O(\sqrt{n})$ iterations. The loop body is O(1), so the function is $O(\sqrt{n})$, a.k.a. $O(n^{½})$.

Basic question 6: Graphs

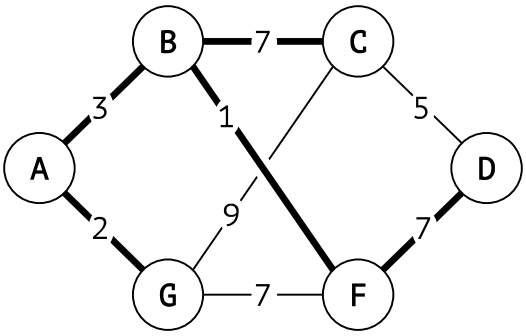
For each of these four graphs, indicate if the highlighted subgraph forms a minimum spanning tree (MST), a shortest-path tree (SPT) starting in vertex A, both of these, or neither.



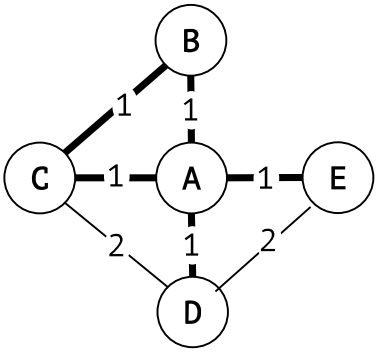
☐ MST ☐ SPT from A



X☐ MST ☐ SPT from A



☐ MST X☐ SPT from A



☐ MST ☐ SPT from A

Advanced question 7: Transactional queue

A *transactional queue* supports the standard enqueue and dequeue operations, but also two additional operations:

- **commit()**: saves the current state of the queue.
- **rollback()**: restores the queue to the exact state it was in the last time commit() was called.

Your task is to design a data structure implementing a transactional queue.

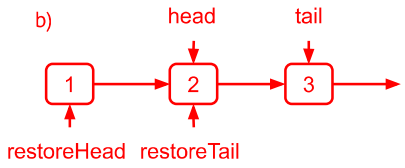
a) Implement the data structure in pseudocode, or describe it in detail. For full points, all operations must be O(1) worst-case time.

b) Draw an illustration of the memory content after performing the following sequence of operations starting from an empty transactional queue:

enqueue(1); enqueue(2); commit(); enqueue(3); dequeue()

a) The following solution extends a linked list queue (with a type of nodes with *value* and *next* fields). The only changes are new fields *restoreHead* and *restoreTail* and new methods *commit* and *rollback*.

b)




```
class TransactionalQueue:
    head : Node
    tail : Node

    restoreHead : Node
    restoreTail : Node

    enqueue(x):                    // standard add-last operation on head/tail
        n = new Node(x, null)
        if head is null:
            head = n
        else:
            tail.next = n
        tail = n

    dequeue():                     // standard remove-first operation on head/tail
        res = head.value
        head = head.next
        return res

    commit():
        restoreHead = head
        restoreTail = tail

    rollback():
        head = restoreHead
        tail = restoreTail
        if head is not null:
            tail.next = null       // cut off any values added after the last commit
```

Advanced question 8: v-leafed minimum spanning trees

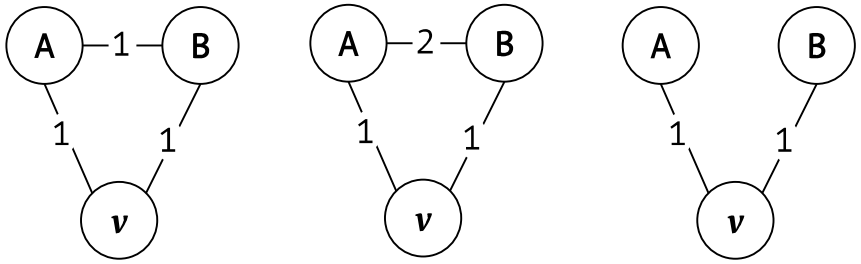
You have a weighted undirected graph G with a specified vertex *v*. A spanning tree of G is considered *v-leafed* if the degree of *v* is 1 in the spanning tree, that is, if there is only one edge to/from *v*.

Your task is to design an algorithm to determine if G has a **minimum** spanning tree (MST) that is *v*-leafed.

You have access to a function mstCost(g) that calculates the total weight of all edges in any MST of a given graph g (think of it as running Kruskal's algorithm and returning only the combined cost of the chosen edges). It returns null if there is no MST, which means that g is not connected.

You are allowed to modify G (adding or removing edges and vertices to the graph).

Here are three tiny example graphs. Only the first of these has a *v*-leafed MST (it even has two, but keep in mind that it also has a non-*v*-leafed MST!). Your algorithm should give true for the first graph, and false for the other two.



Answer (explain your algorithm with code and/or text and figures):

Solution:

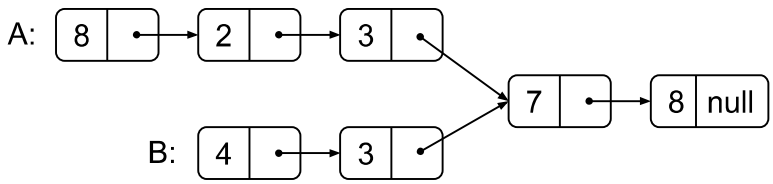
Idea:
Two observations:
- If there is an MST where v is a leaf, then it always contains the shortest edge from v.
- If the graph becomes disconnected by removing v, there is no v-leafed MST.
We calculate the MST cost of G, then the MST cost of G with v removed+the shortest edge from v. The first is the cost of every MST of G, and the second the minimal possible spanning tree where v is a leaf. If and only if the two match, there is an MST where v is a leaf.

```
hasVleafedMST(G, v):
    fullMstCost = mstCost(G)
    if fullMstCost is null: return false      // G has no MST
    minWeightFromV = the weight of the shortest edge from v
    remove v from G
    mstCostWithoutV = mstCost(G)
    if mstCostWithoutV is null: return false   // G is disconnected without v
    return mstCostWithoutV+minWeightFromV == fullMstCost
```

Advanced question 9: Shared tail of two linked lists

Consider two singly-linked lists that may share a common sequence of nodes. If the lists intersect, they share the **same node objects** from the intersection point onward. Cycles are not allowed.

For example, the two lists A=[8,2,3,7,8] and B=[4,3,7,8] could look like this in memory:



In this example, the node containing 7 is the first shared node.

Assume the following standard definition of nodes:

```
class ListNode:
    value: Integer
    next: ListNode      // can be null
```

Your task is to implement this method:

```
findIntersection(A: ListNode, B: ListNode) → ListNode
```

The method must return a reference to the **first node shared by the lists** A and B. If the lists do not intersect, return `null`.

Hint: Use "x is y" in pseudocode to check if ListNodes x and y are the same objects in memory (as opposed to just having identical values). You don't have to access the values at all!

Requirements

Your solution must satisfy all of the following:

- Time complexity: **O(n + m)**, where n and m are the lengths of the two lists.
- Extra space complexity: **O(1)**. You may not use arrays, HashSet, HashMap, or any other auxiliary data structure whose size depends on the input.
- You may not modify either linked list.

Here is a clever solution. It uses two pointers *x* and *y*, initially positioned at the heads of the two lists. When a pointer reaches the end of its current list, we redirect it to the head of the other list. So:

- *x* traverses the list starting at *A* and then the list starting at *B*,
- *y* traverses the list starting at *B* and then the list starting at *A*.

This compensates for any difference in the lengths of the two lists: in the second part of the traversals, both pointers are equally far from the end, hence equally far from the start of any

shared tail. If no shared node is found, both pointers simultaneously become null after finishing both traversals.

```
findIntersection(A: ListNode, B: ListNode) → ListNode:
    x = A
    y = B
    until x is y:
        x = x.next if x not null else B
        y = y.next if y not null else A
    return x
```

There are many alternative solutions. The idea of the following solution is to reduce to the case that the two lists have equal length. In that case, we can go through them in tandem and test for reference/pointer equality at every step:

```
length(x: ListNode) → Integer:
    r = 0
    while x not null:
        x = x.next
        r = r + 1
    return r

findIntersection(A: ListNode, B: ListNode) → ListNode:
    lA = length(A)
    lB = length(B)
    repeat max(0, lA − lB) times: A = A.next
    repeat max(0, lB − lA) times: B = B.next
    // now A and B have the same length
    until A is B:
        A = A.next
        B = B.next
    return A
```

We can implement the reduction without arithmetic as follows:

```
findIntersection(A: ListNode, B: ListNode) → ListNode:
    x = A, y = B
    while x and y not null: x = x.next, y = y.next
    while x not null: x = x.next, A = A.next
    while y not null: y = y.next, B = B.next
    // now A and B have the same length
    until A is B: A = A.next, B = B.next
    return A
```

This is quite close to the clever solution. It uses *A* and *B* instead of *x* and *y* for the second part of the traversals. Performance-wise, they do the same number of assignments. The clever solution does more checks, except when *A* and *B* have the same length, in which case those checks allow it to avoid the second part of the traversals.